Puppet SELinux Boolean Module

RHEL 8 / 9 / 10 - custom OpenVox/Puppet module documentation

Purpose: Manage active and persistent SELinux boolean settings from OS-keyed Hiera data on RHEL 8, 9, and 10.

Module layout

```
modules/custom/
  manifests/selinux_boolean.pp
  files/selinux/selinux_boolean.rb
  functions/source_candidates.pp
  manifests/rpm_install.pp
  data/common.yaml
```

Configuration

```
# Environment Hiera; examples, not prescribed policy.
classes:
  - custom::selinux_boolean

custom::selinux_boolean::set_booleans:
  '8':
    httpd_can_network_connect: 'on'
    httpd_enable_homedirs: 'off'
  '9':
    httpd_can_network_connect: 'off'
    httpd_enable_homedirs: 'on'
  '10':
    httpd_can_network_connect: 'off'
```

Behavior and safety: Only the current RHEL major version is selected. Hiera deep-merges individual boolean entries; missing OS entries manage nothing. Quote OS keys and on/off values. The helper checks active and saved values, runs setsebool -P only on drift, and verifies convergence. Disabled SELinux, unknown booleans, and command failures fail the resource. Required RHEL packages are installed or reused from rpm_install; conflicting xrpms fail compilation. No external Puppet module is required. Removing an entry leaves it unmanaged, and an empty hash does not erase inherited entries.

Supported platform: Red Hat Enterprise Linux 8, 9, and 10. Generated from the repository sources; rerun **tools/generate_custom_module_docs.ps1** after code changes.

SELinux Boolean

modules/custom/manifests/selinux_boolean.pp

```
# @summary
#   Persistently manages SELinux booleans selected by RHEL major version.
# @param set_booleans
#   OS major -> boolean name -> quoted 'on' or 'off'. Deep-merged by Hiera.
class custom::selinux_boolean (
  Hash[Enum['8', '9', '10'], Hash[Pattern[/\A[a-zA-Z][a-zA-Z0-9_]*\z/], Enum['on', 'off']]] $set_booleans = {},
) {
  if $facts['os']['family'] != 'RedHat' or $facts['os']['release']['major'] !~ /\A(8|9|10)\z/ {
    fail('custom::selinux_boolean supports only RHEL 8, 9, and 10')
  }

  $os_major = $facts['os']['release']['major']
  $selected = $set_booleans[$os_major] ? {
    undef   => {},
    default => $set_booleans[$os_major],
  }

  if $selected != {} {
    include custom::rpm_install

    $required_packages = ['libselinux-utils', 'policycoreutils', 'policycoreutils-python-utils']
    $required_packages.each |String $package_name| {
      if $package_name in $custom::rpm_install::xrpms {
        fail("custom::selinux_boolean requires ${package_name}; it cannot be in custom::rpm_install::xrpms")
      }
      if !($package_name in $custom::rpm_install::rpms) {
        package { $package_name:
          ensure => installed,
        }
      }
    }

    file { 'custom SELinux boolean helper':
      ensure       => file,
      path         => '/usr/local/sbin/ces-selinux-boolean',
      owner        => 'root',
      group        => 'root',
      mode         => '0755',
      source       => custom::source_candidates('puppet:///modules/custom/selinux/selinux_boolean.rb'),
      sourceselect => 'first',
    }

    $selected.each |String $boolean_name, String $value| {
      exec { "custom SELinux boolean ${boolean_name}":
        command   => "/opt/puppetlabs/puppet/bin/ruby /usr/local/sbin/ces-selinux-boolean --apply ${boolean_name} ${value}",
        unless    => "/opt/puppetlabs/puppet/bin/ruby /usr/local/sbin/ces-selinux-boolean --check ${boolean_name} ${value}",
        timeout   => 300,
        logoutput => 'on_failure',
        require   => [File['custom SELinux boolean helper'], Package[$required_packages]],
      }
    }
  }
}
```

SELinux Boolean

modules/custom/files/selinux/selinux_boolean.rb

```
#!/opt/puppetlabs/puppet/bin/ruby
# frozen_string_literal: true

require 'open3'

# semanage reports (active, saved) values. Checking getsebool alone would miss
# a matching active value that will revert after reboot.
class CesSelinuxBoolean
  def initialize(runner = nil)
    @runner = runner || lambda do |*argv|
      stdout, stderr, status = Open3.capture3({ 'LC_ALL' => 'C' }, *argv)
      raise "#{argv.first} failed: #{stderr.strip}" unless status.success?

      stdout
    end
  end

  def validate(name, value)
    raise 'Invalid SELinux boolean name' unless name.is_a?(String) && name.match?(/\A[a-zA-Z][a-zA-Z0-9_]*\z/)
    raise "SELinux boolean value must be 'on' or 'off'" unless %w[on off].include?(value)
  end

  def matches?(name, value)
    validate(name, value)
    @runner.call('/usr/sbin/selinuxenabled')
    output = @runner.call('/usr/sbin/semanage', 'boolean', '--list', '--noheading')
    pattern = /\A#{Regexp.escape(name)}\s+\(\s*(on|off)\s*,\s*(on|off)\s*\)/
    match = output.each_line.filter_map { |line| pattern.match(line) }.first
    raise "SELinux boolean #{name} was not found in the installed policy" unless match

    match[1] == value && match[2] == value
  end

  def apply(name, value)
    return if matches?(name, value)

    @runner.call('/usr/sbin/setsebool', '-P', name, value)
    raise "SELinux boolean #{name} did not converge" unless matches?(name, value)
  end
end

if $PROGRAM_NAME == __FILE__
  begin
    mode, name, value = ARGV
    unless ARGV.length == 3 && %w[--check --apply].include?(mode)
      raise 'Usage: ces-selinux-boolean --check|--apply BOOLEAN on|off'
    end

    manager = CesSelinuxBoolean.new
    if mode == '--check'
      exit(manager.matches?(name, value) ? 0 : 1)
    end
    manager.apply(name, value)
  rescue StandardError => e
    warn "SELinux boolean: #{e.message}"
    exit 2
  end
end
```

```
# Returns network node, project, role, network default, then shared default.
# Overrides are dot-suffixed files beside the default (see NETWORK_OVERRIDES.md).
function custom::source_candidates(String[1] $default_source) >> Array[String] {
  $network = $trusted['extensions']['pp_network']
  $project = $trusted['extensions']['pp_project']
  $role = $trusted['extensions']['pp_role']
  $certname = $trusted['certname']

  if $network == undef or $network == '' {
    [$default_source]
  } else {
    # Reject path traversal and URI delimiters in filename suffixes.
    [$network, $project, $role, $certname].each |$identity| {
      if $identity != undef and $identity != '' {
        if $identity !~ String or $identity !~ /\A[A-Za-z0-9][A-Za-z0-9_.-]*\z/ {
          fail('custom::source_candidates identities must start with a letter or digit and contain only letters, digits, underscores, dots, or hyphens')
        }
      }
    }

    $network_source = "${default_source}.${network}"
    $node_sources = $certname ? {
      undef   => [],
      ''      => [],
      default => ["${network_source}.${certname}"],
    }
    $project_sources = $project ? {
      undef   => [],
      ''      => [],
      default => ["${network_source}.${project}"],
    }
    $role_sources = $role ? {
      undef   => [],
      ''      => [],
      default => ["${network_source}.${role}"],
    }

    $node_sources + $project_sources + $role_sources +
      [$network_source, $default_source]
  }
}
```

```
# @summary
#   Installs RPM packages supplied through Hiera.
#
# @param rpms
#   Package names to install. Hiera merges this array uniquely across all
#   hierarchy levels.
# @param xrpms
#   Package names to remove. Hiera merges this array uniquely across all
#   hierarchy levels.
class custom::rpm_install (
  Array[String[1]] $rpms = [],
  Array[String[1]] $xrpms = [],
) {
  if $facts['os']['family'] != 'RedHat' {
    fail('custom::rpm_install supports only Red Hat family operating systems')
  }

  $conflicting_rpms = $rpms.filter |String[1] $rpm| {
    $rpm in $xrpms
  }
  if $conflicting_rpms != [] {
    fail("custom::rpm_install cannot install and remove the same packages: ${conflicting_rpms.join(', ')}")
  }

  $rpms.each |String[1] $rpm| {
    package { $rpm:
      ensure => installed,
    }
  }

  $xrpms.each |String[1] $rpm| {
    package { $rpm:
      ensure => absent,
    }
  }
}
```

```
---
lookup_options:
  custom::selinux_boolean::set_booleans:
    merge:
      strategy: deep
  custom::rpm_install::rpms:
    merge:
      strategy: unique
  custom::rpm_install::xrpms:
    merge:
      strategy: unique

custom::rpm_install::rpms: []
custom::rpm_install::xrpms: []
custom::selinux_boolean::set_booleans: {}
```